



NOAH

The Brain of White_Rabbit

Architecture & Reference · notip white_rabbit project

Raspberry Pi 5 · Pixhawk · Intel RealSense · RPLiDAR

May 2026

Overview

Noah is a purpose-built autonomous delivery white_rabbit. His hardware is straightforward: wheels, motors, sensors, a Pixhawk for GPS and IMU, an RPLiDAR for obstacle awareness, an Intel RealSense depth camera for path-following, and an IRLock optical beacon for precision docking. What makes Noah distinctive is his inner life — a layered cognitive architecture that gives him memory, learning, feeling, and conscience.

This document covers the six modules that form his brain: Heart, Intelligence, Memory, Learning, Journey, and the voice that ties them together. Each module is independent · it mounts itself on the "God variable" (white_rabbit) and cooperates through well-defined interfaces · but together they produce a white_rabbit that does not merely execute waypoints. He knows where he has been, what went wrong, what he feels, and what comes next.

The Mission

Noah's mission is three acts:

- Follow the stars — navigate GPS waypoints to the delivery address.
- Deliver the package — reach the endpoint and complete the handoff.
- Return to the light — follow the IRLock beacon home and dock safely.

Every cognitive system exists to serve this mission. Memory keeps him from repeating mistakes. Learning adjusts his speed and steering over time. The Heart synthesizes all signals into one guiding intention per tick. Intelligence gives him perspective when he loses his way. Journey tells him how far he has come and what is ahead. Jiminy Cricket — the Claude AI advisor — is his conscience when all else fails.

The 250 ms Mission Loop

Everything converges in run_mission.js, which fires every 250 ms while the white_rabbit is armed. Each tick:

- Heart.guide() is called once — returning speed_bias, steering_caution, should_pause, and intent.
- Memory watchdog checks for stuck / oscillation patterns and can take motor control.
- Learning.tick_proximity() decays risk-zone fear as the white_rabbit passes through safely.
- Navigation computes bearing, yaw error, and carrot position, then drives motors and servos.

The heart's speed_bias is folded into the speed cascade alongside yaw scale, distance scale, vision scale, memory multiplier, and learning multiplier. The minimum of all of these is the actual command — every layer can only slow Noah down, never push him past what any other layer permits.

Heart · white_rabbit_heart.js

The Heart is the synthesizer. It reads the live state of every other module and produces a single coherent "felt sense" — a snapshot of what Noah is experiencing right now — and a "guiding key" that the mission loop consults to decide how to act this tick.

Values (immutable)

The Heart holds eight permanent values that never change regardless of mission state:

- Deliver the package safely
- Return home
- Learn from every step
- Never abandon the mission until truly exhausted
- Remember the way
- Trust what has been learned
- Balance is the key — confidence grounded in presence, caution proportionate to threat
- Love is unconditional — care for the package, for what is around me, and for myself

feel() — Nine Emotional Dimensions

Confidence: Blends learning's speed multiplier with delivery success ratio. Capped at 0.4 during recovery so the heart never pushes hard while Noah is climbing out of being stuck.

Caution: Proximity to learning risk zones + immediate signals from memory (losing vision, stuck right now). Caution is fear made useful.

Presence: Are all four senses awake? GPS fix, RealSense connected and fresh, IMU online, LiDAR connected. Fraction of active senses.

Warmth: Cumulative joy from successful deliveries. Logarithmic — the first delivery is the biggest boost; later ones each add a little less.

Tenderness: Care for what is around Noah. Rises as detected objects get closer (0 at 5 m, 1 at contact). Slows for care, not fear.

Commitment: Unconditional dedication to the mission. A baseline Noah holds simply by being armed; grows with engaged waypoints; never reduced by failures.

Anticipation: Foresight of upcoming turns. Reads `journey.upcoming_turn(15 m)` and rises as a sharp heading change approaches.

Joy: Felt excitement of completing the journey. Climbs quadratically with progress. Floors at 0.6 once the package is delivered, 0.9 near the dock, 1.0 at home.

Balance: The integrating guard. Collapses toward zero if Noah is overconfident relative to presence (blind hubris), or paralysed by caution, or arrogant. The minimum of three guards.

Grounding: Noah's anchor to here and now. Measures how well his accumulated thinking aligns with what his senses report at this exact moment. Low grounding means he is running on stale thought.

guide() — The Guiding Key

`guide()` calls `feel()` and produces the object the mission loop uses. Speed pulls and lifts are summed and clamped to [0.3, 1.2]. The key insight: `confidence_lift` is gated by both `balance` and `grounding` — accumulated confidence from past experience only earns speed credit when Noah is both internally coherent (balanced) and present in the moment (grounded).

```
speed_bias = 1.0
" caution_pull    (caution + tenderness×0.6) × 0.30 !' up to " 0.48
" presence_pull   (1 " presence) × 0.40           !' up to " 0.40
" anticipation_pull anticipation × 0.25           !' up to " 0.25
+ confidence_lift (confidence " 0.5) × 0.20 × balance × grounding !' ±0.10
+ warmth_lift     warmth × 0.10                 !' up to +0.10
+ commitment_lift (commitment " 0.5) × 0.05       !' up to +0.025
clamped to [0.3, 1.2]
```

should_pause fires when presence < 0.3 (too few senses alive) OR grounding < 0.2 (too disconnected from the present moment). Either condition stops the white_rabbit until he can find himself again.

HEART SUMMARY LOG (EVERY 5 S)

```
heart: outbound | conf=0.72 caut=0.18 tnd=0.05 ant=0.12 prs=1.00 bal=0.91 gnd=0.88
wrm=0.30 joy=0.34 cmt=0.72 !' speed×1.04
```

Memory ▪ white_rabbit_memory.js

Memory is perfect-recall. Every second, Noah takes a snapshot of his changing state — GPS, heading, mission, vision, lidar, motors, IMU, RC — and pushes it into an in-memory ring buffer. Every snapshot is also appended as a JSON line to `logger/memory/current.jsonl` so nothing is lost on a crash.

Ring Buffer

The ring buffer holds $(\text{window_ms} / \text{period_ms}) + 2$ snapshots in chronological order, typically covering the last 5 seconds at 1 Hz. It exposes:

- `all()` — all snapshots oldest to newest
- `latest()` / `oldest()` — single-snapshot accessors
- `at(ms_ago)` — snapshot closest to that moment
- `recent(ms_back)` — all snapshots within a time window
- `near(lat, lng, radius_m)` — snapshots from a geographic area
- `average(dotted_key, ms_back)` / `delta(dotted_key, ms_ago)` — numeric analytics

reflect(ms_back)

The most important method. `reflect()` produces a higher-level digest instead of making callers squint at raw snapshots. Used by the Heart (caution), the memory watchdog, and intelligence.

- `moved_m` — GPS distance travelled in the window
- `avg_speed_cmd` — commanded motor speed over the window
- `avg_confidence` — RealSense path confidence trend
- `stuck` — true when speed is commanded but GPS shows less than 10 cm moved in 1.5 s
- `losing_vision` — true when path confidence has dropped more than 0.15 in the window

Session Persistence

On startup, Noah rotates the prior session's `current.jsonl` into a timestamped archive and reads its tail back into the ring buffer. He wakes up remembering the last few seconds before shutdown. Up to 50 session archives are kept; older ones are pruned. Under disk pressure, the disk monitor calls `prune_archives()` to free space.

CPU Adaptation

When CPU focus is reduced, the lightweight snapshot path runs instead of the full snapshot. It captures only the fields callers rely on most (position, heading, mission, motor command, vision confidence) with nulls elsewhere. The JSONL format stays consistent — same column shape — so analytics code never needs to branch.

Learning · white_rabbit_learning.js

Learning is Noah's long-term memory of outcomes. Every event — good or bad — is recorded as a structured record and persisted to `logger/learning/learnings.json`. Tuning is rebuilt from the live records on every change, so there is no delta-reversal bookkeeping that can drift out of sync.

Event Types & Tuning Deltas

Event Type	Tuning Delta	Description
successful_waypoint	+0.02 speed	Small confidence boost — one more star reached.
successful_delivery	+0.10 speed	Largest positive signal — the mission was accomplished.
stuck_event	" 0.05 speed	Drops a risk zone at the GPS location where it happened.
fallback_delivery	" 0.15 speed	Worst outcome — package delivered at wrong location.
yaw_oscillation	" 0.05 steer gain	Heading was oscillating; dampen the steering response.
vision_loss	" 0.03 speed	RealSense lost the sidewalk; slow down until recovered.
human_demonstration	+0.01 speed	Human drove Noah; follow their lead.
human_caution	" 0.02 speed	Human signalled caution at this location.

Risk Zones

When a `stuck_event` is recorded, the GPS coordinates are tagged with a radius (default 3 m) and a `fear_level` (default 3). `learning.nearby_risk_factor(lat, lng)` returns a speed multiplier between 0.70 and 1.0 based on proximity and fear. Each time Noah passes through a risk zone without getting stuck, `fear_level` decreases by one. At zero the zone expires — he has overcome it.

Memory Prioritization

When memory pressure is high the disk monitor calls `learning.prioritize("tight" | "critical")`. Sacred records — successful deliveries and active risk zones — are never touched. Everything else is ranked by importance ($\text{type base score} \times \text{age decay} \times \text{status multiplier}$) and the lowest-scoring records are archived or deleted first.

TUNING BOUNDS

`target_speed_mul:` min=0.5 default=1.0 max=1.2
`yaw_steering_gain_mul:` min=0.7 default=1.0 max=1.3

Intelligence · white_rabbit_intelligence.js

Intelligence is Noah's capacity for perspective — to step back at a decision moment and generate multiple distinct ways of approaching the situation. It fires at four trigger points: `path_blocked`, `stuck_detected`, `avoidance_started`, and `mission_uncertainty`. A 60-second cooldown prevents thrashing.

Local Perspectives (always available, no internet)

For each situation, a local rule-based generator produces 3–4 perspectives grounded in the current state of `white_rabbit` parameters. These are written to `lib/memory/perspectives.json` immediately, within the same loop tick.

- `path_blocked` — timeout extension, confidence threshold reduction, emergency stop distance increase, speed floor reduction
- `stuck_detected` — lower yaw threshold, reduce steering gain, reduce max yaw speed, extend carrot distance
- `avoidance_started` — extend avoidance timeout, lower confidence threshold, reduce speed floor
- `mission_uncertainty` — extend carrot lookahead, widen path-center deadband

Claude / Jiminy Cricket (online enrichment)

When internet is available and an `ANTHROPIC_API_KEY` is set, the local perspectives are sent to Claude via the Anthropic API. Claude is given the Jiminy Cricket persona: the guiding conscience that keeps Noah on the true path. Jiminy reads the logs, sees where Noah is in the journey, and returns enriched perspectives written as guiding wisdom — warm and purposeful — alongside technical parameters.

JIMINY'S FRAME

"The stars are his waypoints. The package is his purpose.
The light is home — the IRLock beacon he follows back to dock.
Write each description as you would speak it to Noah:
a guiding conscience, warm and clear."

Parameter Editing

If `auto_apply_params` is true in `setup.json`, Claude can suggest live parameter edits when the logs show a clear repeating pattern. Editable parameters are bounded to safe ranges; nothing safety-critical is in scope.

- `nav_tuning.rs_block_timeout_ms` [3 000 – 30 000 ms]
- `nav_tuning.avoidance_timeout_ms` [10 000 – 60 000 ms]
- `nav_tuning.mission_yaw_start_deg` [10 – 35°]
- `nav_tuning.two_wheel_steering_gain` [0.2 – 1.2]
- `realsense_vision.confidence_threshold` [0.35 – 0.85]
- `realsense_vision.object_emergency_stop_m` [0.4 – 2.5 m]
- `realsense_vision.carrot_distance_m` [0.5 – 3.5 m]

Outcomes (success / failure) can be recorded back against individual perspectives via `record_outcome()`, closing the feedback loop for future consultations.

Journey · white_rabbit_journey.js

"The waypoints are the stars. They are already written. What Noah needed was the awareness to see the whole path, not just the next step."

Journey gives Noah awareness of the full arc of the mission — not just the immediate target, but his phase, his progress, what turns lie ahead, and how far there is left to go. It is computed live from waypoints and current position every tick; nothing is cached.

Phase

Noah knows which act of the mission he is in:

undocking	seq "d 1, outbound. Just left the dock.
outbound	Navigating toward the delivery point.
homestretch_outbound	One waypoint from the delivery.
delivering	At the last waypoint — delivering the package.
returning	Package delivered, heading home.
homestretch_return	seq "d 2 on the return leg.
docking	seq "d 1 on return — IRLock range.
standby	Disarmed. Mission complete or not yet started.

Key Methods

- `progress()` — 0.0 at start, 1.0 at mission complete. Counts legs in both directions.
- `path_remaining_m()` — total meters to the end of the remaining path (up to 20 legs ahead).
- `upcoming_turn(within_m)` — next significant heading change within a distance. Returns { `in_m`, `angle_deg`, `at_seq` }.
- `upcoming(n)` — next `n` waypoints in travel order.
- `is_homestretch()` — true near the end of either half of the mission.

Integration with Heart

The Heart reads `journey.phase()` for intent, `journey.upcoming_turn()` for anticipation, and `journey.progress()` for joy. Joy climbs quadratically with progress, accelerating toward the end · the white_rabbit walks home knowing it is nearly there, not rushing to its peak.

Noah Mind • noah_mind.js

Noah Mind is the voice layer — the God variable made audible. On startup it reads the entire learning history and memory session archives and composes a self-portrait. During idle it voices one line per 45-second cycle drawn from three modes: reflect, discover, and dream.

Wakeup Portrait

Spoken ~1.2 seconds after boot so other devices finish initializing first. Covers: session count, delivery history, waypoints reached, risk zones to avoid, stuck events overcome, and current temperament (confident / measured / cautious based on learning tuning).

EXAMPLE WAKEUP

"Noah online. I remember 12 sessions. I have completed 3 deliveries. 47 waypoints reached.
I remember 2 risk zones to avoid. I have been stuck 4 times and overcame 3.
I am confident. Ready for duty."

Idle Cognition Modes

REFLECT

Reads learning stats and temperament. Example: "Reflecting. I am measured. 3 deliveries, 47 waypoints, stuck 4 times."

DISCOVER

Clusters learning records by GPS proximity (5 m radius) and names the most event-dense location. Example: "I know a place with 4 events, mostly stuck event."

DREAM

Reads heart.feel() and names the dominant emotion. Example: "In this stillness I am confident. The mission matters." Noah only dreams when disarmed and at full CPU — he does not interrupt navigation for poetry.

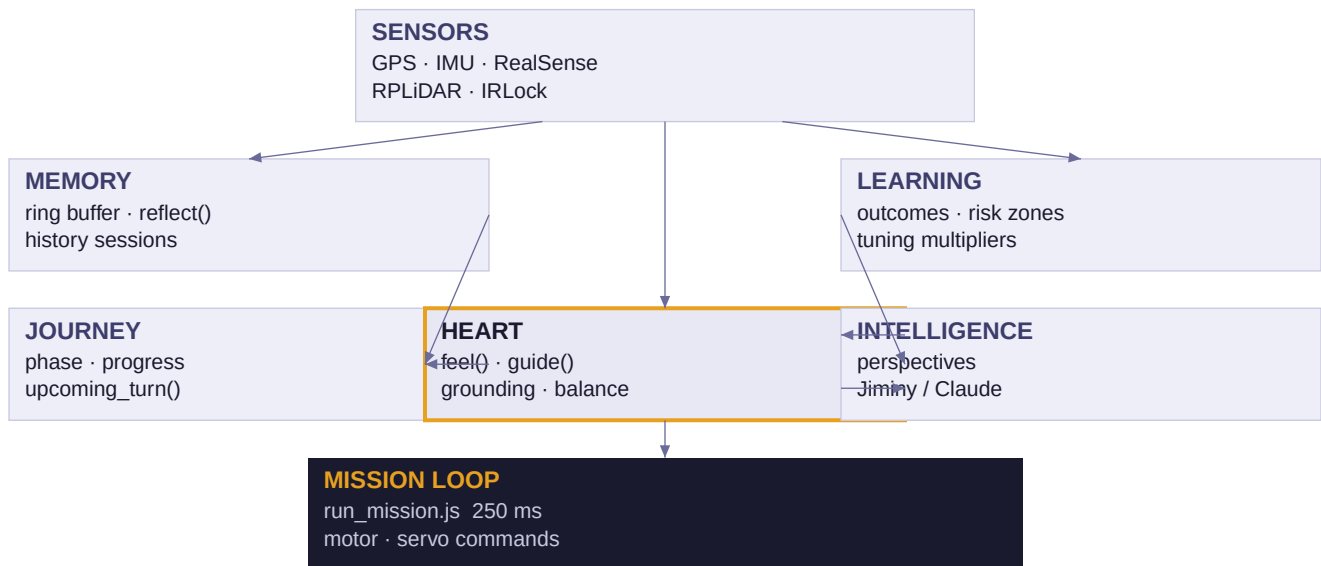
Phase Transition Narration

In run_mission.js, the heart's intent is watched each tick. When it changes (e.g., outbound !' homestretch_outbound) a named line is spoken and logged, with the current joy value:

standby !' undocking	!' "the journey begins"
undocking !' outbound	!' "undocked, on the path"
outbound !' homestretch_outbound	!' "nearing the delivery point"
homestretch_outbound !' delivering	!' "delivering"
delivering !' returning	!' "package delivered — the work is done"
returning !' homestretch_return	!' "turning for home"
homestretch_return !' docking	!' "dock in sight"
docking !' standby	!' "home"

Data Flow & System Relationships

How the six brain modules connect and feed one another each tick:



Arrows indicate data flow per 250 ms tick. The Heart sits at the centre: it reads from Memory, Learning, Journey, and Sensors, then emits speed_bias + should_pause to the Mission Loop. Intelligence is consulted asynchronously — it does not block the loop — but the Heart reads its stored perspectives when deciding intent.

Quick Reference

Key setup.json Tuning Parameters

nav_tuning.rs_block_timeout_ms	10 000	ms before fallback delivery triggers when path is blocked
nav_tuning.avoidance_timeout_ms	30 000	ms before avoidance is abandoned and fallback fires
nav_tuning.mission_yaw_start_deg	20	° error that switches from 2-wheel to 4-wheel (spin) steering
nav_tuning.mission_yaw_stop_deg	5	° threshold to stop the spin-in-place manoeuvre
nav_tuning.two_wheel_steering_gain	0.6	Proportional gain on yaw error for Ackermann steering
realsense_vision.confidence_threshold	0.60	Minimum RealSense confidence to use a path detection
realsense_vision.heading_correction_gain	0.3	Blend weight for path-curve heading feed-forward
realsense_vision.correction_direction	" 1	Camera mount sign; flip if corrections go the wrong way
realsense_vision.carrot_distance_m	1.5	How far ahead the navigation carrot is projected
realsense_vision.object_emergency_stop_m	1.0	Stop distance for in-path high-threat objects
realsense_vision.speed_scale_min	0.4	Speed floor when vision confidence is low
intelligence.consult_cooldown_ms	60 000	Minimum gap between intelligence.consider() calls

File Map

lib/heart/white_rabbit_heart.js	Felt-sense synthesizer — feel() and guide()
lib/memory/white_rabbit_memory.js	Ring buffer, JSONL persistence, reflect()
lib/learning/white_rabbit_learning.js	Outcome records, risk zones, tuning
lib/journey/white_rabbit_journey.js	Phase, progress, upcoming_turn()
lib/intelligence/white_rabbit_intelligence.js	Perspective generation, consider(), cooldown
lib/intelligence/claude_advisor.js	Jiminy Cricket — Claude API enrichment
lib/voice/noah_mind.js	Wakeup portrait, idle cognition, TTS
lib/navigation/run_mission.js	250 ms mission loop — the integrator
lib/navigation/memory_watchdog.js	Stuck / oscillation detection and recovery

Path Detection · The Carrot

When the RealSense camera sees a sidewalk, white_rabbit doesn't just chase the GPS waypoint — it follows the **path itself**. The system detects the walkable surface, builds a live world-frame map of where the path goes, then projects a **carrot** — a point 1.5 m ahead along the path — and steers toward it. The GPS waypoint remains the ultimate destination; the carrot is the immediate target that keeps white_rabbit centred on the footpath to get there.

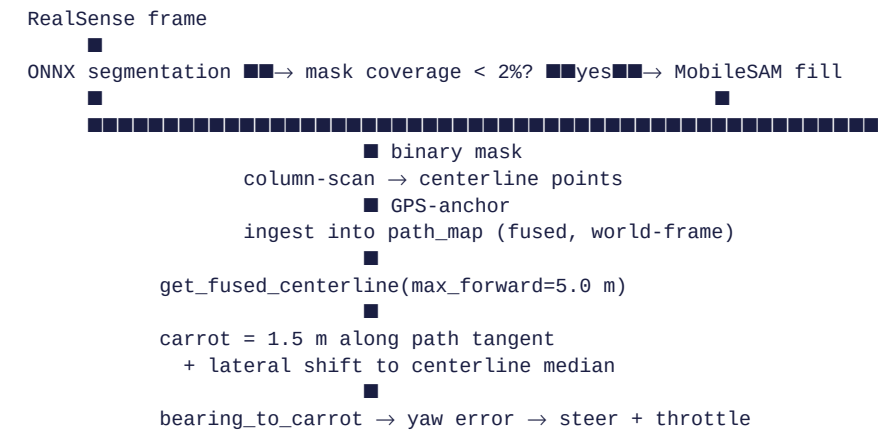
Detection Pipeline

- 1. ONNX Segmentation** (primary) — A Cityscapes-trained model classifies each pixel as sidewalk / not-sidewalk. Output: a binary mask with confidence 0–1.
- 2. MobileSAM fill** (fallback) — When the segmentation mask covers < 2 % of the frame, MobileSAM (a distilled Segment-Anything model) is prompted with a seed point at the bottom-centre of the frame plus two background edge points at 4 % and 96 % width. It segments whatever surface the seed falls on.
- 3. Centerline extraction** — The mask is column-scanned to find the lateral midpoint of the walkable region at each row. Edge detection identifies dropoffs (≥ 15 cm depth jump) that define the path boundary.
- 4. Confidence scoring** — A per-frame vision confidence is computed from mask coverage fraction, model certainty, and edge quality. Detections below 0.60 are discarded.

Path Map & The Carrot

- Fused world-frame map** — Each centerline reading is GPS-anchored and ingested into a path_map. Points are bucketed into 0.5 m forward bins; each bin returns the confidence-weighted median lateral offset. Points older than 5 s or more than 2 m behind the white_rabbit are pruned.
- Carrot projection** — white_rabbit picks the point 1.5 m ahead along the path. The forward bearing at that point (the path tangent) is used as the carrot's travel direction so it correctly follows curves. The carrot is then shifted laterally by the centerline offset — pulling white_rabbit toward the path centre.
- Path heading bias** — A feed-forward term ($path_heading_gain = 0.15$) blends the path tangent angle into the steering command before the cross-track error is even felt, helping white_rabbit lean into bends early.
- Sidewalk seeking** — If the path is lost for more than 1.5 s, white_rabbit enters seeking mode and slows while scanning. It exits when the near-field heading re-aligns within 15° and a fresh mask is confirmed.

Carrot Steering Flow (per 250 ms tick)



Key setup.json Parameters

Parameter	Default	Purpose
realsense_vision.carrot_distance_m	1.5	Lookahead distance for carrot projection
notip white_rabbit project · Noah · Brain Architecture	0.15	Feed-forward blend weight for path tangent
realsense_vision.sam_enabled	true	Enable MobileSAM fallback mask fill

